As we'll see in the next section, lists can be optimal for insertions and deletions while they are slower for random access.

C.1.2 Linked List

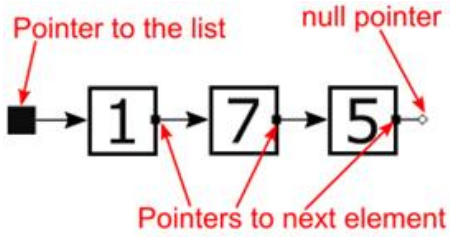
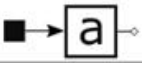


Figure C.3 The graphic representation of a linked list. Here each element holds a pointer (aka reference or link) to the next node in the list. The reference to the first node in held by a special pointer, the head of the list. To go through the whole list, we need to keep the head pointer, and then traverse the list node by node.

A *linked list* stores a sequence of items by wrapping each item in an object, called a node. As it’s shown in figure C.3, each node holds a value and either one or two links (references) to other nodes. A value can be a simple type, like a number, or a complex type, like an object. The order of the elements is determined exclusively by the list's links. Nodes don't need to be allocated contiguously and therefore lists are dynamic by nature. So, they can grow or shrink as needed.

More formally, a list can be defined recursively: it can be either

- An empty list.
- A node containing a value and a reference to a linked list.

Empty List	[]	
Singleton	a::[]	
2 elements	a::(b::[])	
n elements	a::(b::(c::...(x::[])))	